搭建学习环境

typescript 是一个js的超集，使js从动态的语言检查转为静态的语言检查，类似于其他编程语言，在编译之前就可以检查出错误

ts会编译成js，所以ts并不是新的语言，

1.安装typescript

```
yarn add typescript 或者 npm i typescript
//安装ts-node
yarn add ts-node 或者 npm install ts-node
//创建 tsconfig.json 文件
tsc --init
//创建ts，使用命令运行
tsc  xxx.ts//编译ts
ts-node xx.ts //编译且运行ts
```

2.数据类型

基本数据类型

```
//booelan布尔
let isRight: boolean = true
//string 字符串
let str: string = `hello, ${username}`
//number 数字
const bin: number = 0b1010
//null & undefined
//null和undefined是任何类型的子类型
let m1: void = undefined; // null不能赋值给void

let m2: any     = undefined;
let m3: unknown = undefined; //可以赋值给unknown

let n2: any  = null;
let n3: unknown = null; //null也可以赋值给unknown
//symbol
let sym2 = Symbol("key");
let sym3 = Symbol("key");

```

```
sym2 === sym3; // false, symbols是唯一的
//void 返回值为空 ，或者表示没有返回值用于没有返回值的函数（一般用于函数）
//any & unknown 顶端
//any和unkown 一样都是顶端元素 是所有元素的父类类型，any可以赋值给其他元素，而unknown不能

let x: any;//或者unknown一样

x = true;
x = 'hi';
x = 3.14;
x = 99999n;
x = Symbol();
x = undefined;
x = null;
x = {};
x = [];
x = function () {};
let a:number=undes2 //any赋值不报错
let b:number=undes //unknown会报错
//never 尾端 表示那些永远不存在的值，没有子类型，其他的类型不能赋值给该类型除了该类型本身

```

注意null和undefind类型：

> null和undefined是所有类型的**子类型**。如果没有开启strictNullChecks模式，null和undefined可以复制给任意数据类型。如果开启strictNullChecks模式（类型检查），undefined就只能赋值给undefined类型，null就只能赋值给null类型。但这种说法也不准确，undefined值和null值允许赋值给顶端类型，同时undefined值也允许赋值给void类型。
>
> /**
>
> - strictNullChecks: true
>
> */
>
> let m1: void = undefined; // null不能赋值给void
>
> let m2: any = undefined; //都可以赋值给任何类型，前提在tsconfig.json中关闭类型检查
>
> let m3: unknown = undefined;
>
> let n2: any = null;
>
> let n3: unknown = null;

虽然any类型是所有类型的父类型，但是TypeScript允许将any类型赋值给任何其他类型，unknown相对来说安全一点

开发时，尽量少使用any，因为any会跳过类型检查，可能会导致编译错误。通过设置 noImplicitAny": true

unknown类型是比any类型更安全的顶端类型，因为unknown类型只允许赋值给any类型和unknown类型，而不允许赋值给任何其他类型，该行为与any类型是不同的

在程序中使用unknown类型时，我们必须将其细化为某种具体类型，缩小类型，我们可以使用typeof、类型断言等方式来缩小未知范围，否则将产生编译错误。

never 尾端

在 TypeScript 中，可以利用 never 类型的特性来实现全面性检查，**使用 never 避免出现新增了联合类型没有对应的实现，目的就是写出类型绝对安全的代码。 使用时应该使用这个类型**

> never类型表示的是那些永不存在的值的类型。never没有子类型，所以，其他类型都不能赋值给never类型，除了never本身。any也不能赋值给never。
> 值会永不存在的两种情况：
> 1. 如果一个函数执行时抛出了
> 异常，那么这个函数永远不存在返回值（因为抛出异常会直接中断程序运行，这使得程序运行不到返回值那一步，即具有不可达的终点，也就永不存在返回了）；
> 2. 函数中执行无限循环的代码（
> 死循环），使得程序永远无法运行到函数返回值那一步，永不存在返回。

引用数据类型

<u>联合数据类型</u>

```
type div= string|number //定义别名
let num:div='str' //使用联合数据类型
//type div= null|HTMLDivElement
let num:div= document.querySelector('#app')
//枚举类型 关键字 enum 定义枚举类型的数据
enum Direction {
```

```
    Up = 1,     // 1
    Down,       // 2
    Left = 10, // 10
    Right,      // 11
}
// 字符串枚举是string类型的子类型
const direction: string = Direction.Up;
const direction: Direction = 'UP';//字符串类型不能赋值给枚举类型
```

自定义类型：通过type关键字可以自定义类型

```
type div= string |number  //定义类型字符串或者数字
```

数组

```
//数组
let arrs: number[] = [1, 2, 3, 4, 5,]
let arr2: (string| number)[] = [1, '2', 3] //可选数组
//类数组
function sums(){
        let args:IArguments=arguments
        console.log(args);
}
const score:[string,number]=['1',2]
//元组可选
type Poples=[string,number,number?,any?,...string[]]
// const score2:Poples=['str',1]
//元组剩余元素
let resTulp:Poples=['str',1,]
```

接口

接口用来描述对象的形状

```
//接口用来描述对象的形状，形态，命名规范 ：以大写的I开头
//普通接口 属性签名
interface Izhuobiao {
    x: string,
    y: string
    z?:string
}
```

```
let p: Izhuobiao = {
    x: '22',
    y: '11'

}
//调用签名
interface IMessage {
    //括号里面左边代表参数 冒号右边代表返回值
    (aaaa: unknown): void
}
const siHi: IMessage = (data) => {
    console.log(data);
}
//构造签名
interface IPerson {
    //new 的方式
    new(pname: string): Object
}
const Person: IPerson = class {
    name: string //必须先定义
    constructor(pame: string) {
        //定义传入的值
        this.name = pame
    }
}



//方法签名
interface IB {
    saihi: (data: string) => void;//字面量
    suc(x: boolean): string//方法名
    fn: { (x: boolean): string }//属性签名和对象
}
//索引签名
interface A {
    [prop: string]: number,
    //其他的应该都是当前的子集否则会报错
    [prop: number]: number,//拿去的索引
```

```

}


```

函数

可选参数

```
function add(x: number, y?: number, z?: number) {//可选的
   return x + (y ?? 0) + (z ?? 0);
}
```

```
//想使用静态属性1
function f(x: number) {
    console.log(x);
}
f.version = '1.0';

let foo: { (x: number): void; version: string } = f;

const version = foo.version;

```